



THE UNIVERSITY
OF QUEENSLAND
A U S T R A L I A

Inferring security-relevant properties from Java documentation

Connor Myers

Bachelor of Engineers (Honours)

Abstract

Start this section on a new page [this template will automatically handle this].

The abstract should outline the main approach and findings of the thesis and normally must be between 300 and 800 words.

Declaration by author

This thesis is composed of my original work, and contains no material previously published or written by another person except where due reference has been made in the text. I have clearly stated the contribution by others to jointly-authored works that I have included in my thesis.

I have clearly stated the contribution of others to my thesis as a whole, including statistical assistance, survey design, data analysis, significant technical procedures, professional editorial advice, financial support and any other original research work used or reported in my thesis. The content of my thesis is the result of work I have carried out since the commencement of my higher degree by research candidature and does not include a substantial part of work that has been submitted to qualify for the award of any other degree or diploma in any university or other tertiary institution. I have clearly stated which parts of my thesis, if any, have been submitted to qualify for another award.

I acknowledge that an electronic copy of my thesis must be lodged with the University Library and, subject to the policy and procedures of The University of Queensland, the thesis be made available for research and study in accordance with the Copyright Act 1968 unless a period of embargo has been approved by the Dean of the Graduate School.

I acknowledge that copyright of all material contained in my thesis resides with the copyright holder(s) of that material. Where appropriate I have obtained copyright permission from the copyright holder to reproduce material in this thesis and have sought permission from co-authors for any jointly authored works included in the thesis.

Chapter 1

Introduction

Static application security testing (SAST) applies static program analysis to the security of software projects. [MORE INFO ABOUT STATIC ANALYSIS HERE](#). Static analysis is performed on the source code, compiled byte-code, or some other representation of the source code [1]. Popular SAST tools include: SonarQube [2], SpotBugs [3], Snyk Code [4], and Facebook’s Infer [5]. SAST tools have become widely used at all stages of the software development life-cycle. Furthermore, they have been demonstrated to be highly effective in identifying defects in code compared to other methods, such as manual inspection [6].

Unfortunately, SAST tools are not perfect. From Rice’s Theorem, it is known that any non-trivial property about programs is reducible to the halting problem [7]. That is, a ”perfect” analysis is not always possible and approximations must be made in order for the analysis to terminate, resulting in an imperfect output. False positives (where safe code is incorrectly labelled as a defect) and false negatives (where the analysis misses a genuine fault) [8] are common in all static analysis tools. False positives are not ideal as manual inspection is required to separate genuine defects from false positives, leading to an increased time and labour cost, and false negatives mean genuine defects remain undetected. To reduce the rate of false negatives, manual configuration of the SAST tools for specific software projects is often required [9].

For example, to perform an effective taint analysis, a custom configuration of sources, sinks and sanitisers for a particular project is often required. Creating this manual configuration is a time-consuming and resource-intensive process. As such, methodologies to automatically label methods as sources, sinks, and sanitisers may enable existing taint analysis tools to operate more effectively. It should also be noted that in cases where a closed-source library is being used in a software project, it may be impossible to determine how the API methods should be labelled for the taint analysis.

Deep learning, a subtopic within the broader machine learning, has recently increased in popularity. What was once a niche topic reserved for academics with access to computational resources far out

of reach by most has become arguably one of the most influential technologies of the modern age. Deep learning has fundamentally changed our everyday life, with applications ranging from targeted advertising to identifying COVID-19 cases from lung CT scans [10]. Natural-language processing (NLP) is one often overlooked application of deep learning, but it has seen tremendous growth in recent years.

NLP has existed in one form or another for decades. In the early days of NLP, work was painstakingly done using handwritten rules before moving to probabilistic methods in the 1990s and then to deep learning in the 2010s [11]. With tech conglomerates such as Google and Microsoft publicly releasing large pre-trained models, NLP has never been more accessible. While machine translation and speech recognition are among the most popular applications of NLP, one application that has not been explored thoroughly in the literature is software security.

While being an informal description of code, Javadoc (i.e., natural language descriptions of code) can contain information relevant to software security. Using powerful pre-trained models such as CodeBERT that understand the semantic connection between code and natural language [12], it may be possible to extract security-relevant properties from Javadocs and use this information to configure existing SAST tools.

1.1 Topic Definition

Static application security testing (SAST) tools often require additional configuration to conduct a practical security analysis. Typically, this configuration is done manually and demands a substantial time, labour, and skill cost to be done effectively. While being an informal natural-language description of a Java method, Javadoc may contain valuable information for the configuration of SAST tools. With powerful pre-trained natural language processing models being made publicly available, it may be possible to train a deep learning model to extract relevant security properties from Javadocs so that information can be used to configure existing SAST tools.

One such pre-trained model of particular relevance is Microsoft's CodeBERT. CodeBERT has been trained on natural language and numerous programming languages (including Java). CodeBERT understands the semantic connection between natural language and programming languages, making it suitable for downstream tasks involving the relationship between natural language and code. [12].

It is hypothesised that CodeBERT can be fine-tuned with additional data to extract relevant security properties from Javadoc. This information can be used to configure existing static analysis tools. For example, taint analysis requires a list of methods labelled as sources, sinks, and sanitisers. Taint analysis tools often have a default configuration with popular and frequently used APIs. However,

security-critical methods are not included in the default configuration, particularly in large projects, which in turn leads to an incomplete analysis being conducted. Manual inspection is required to label additional methods and add them to the taint analysis configuration. Naturally, this is a labour-intensive process with high-skill requirements. Furthermore, in cases where a closed-source library is being used — where only the binary and documentation are available — it may be difficult or impossible to label these methods correctly. With sufficient training, it is believed codeBERT can adequately label security sensitive methods.

Chapter 2

Deep Learning

To provide context for this project, a high-level overview of deep learning will be provided.

2.1 Deep Learning Model Architectures

There are a large variety of deep learning models, each designed for different purposes and with different strengths and weaknesses. Given how computationally expensive training and fine-tuning models is, ensuring that an appropriate architecture is used (indirectly chosen when selecting a pre-trained model) is an important decision key for this project's success. In this section, we will discuss and compare common deep learning model architectures at a high-level in order to determine an appropriate selection for this project.

2.2 Simple Neural Network

Here, we will outline a basic neural network model to familiarise ourselves with the terminology that will be used later in the report, as well as the project moving forward.

Figure 2.1: A Simple Neural Network

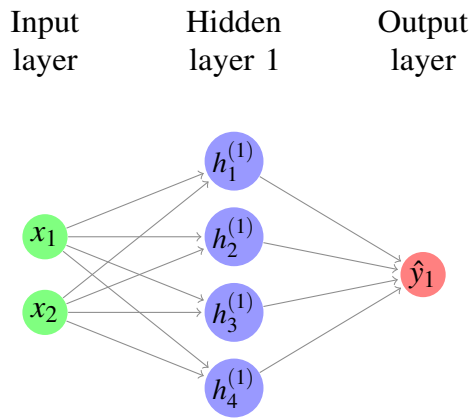


Figure 2.2: Diagram of a singular neuron

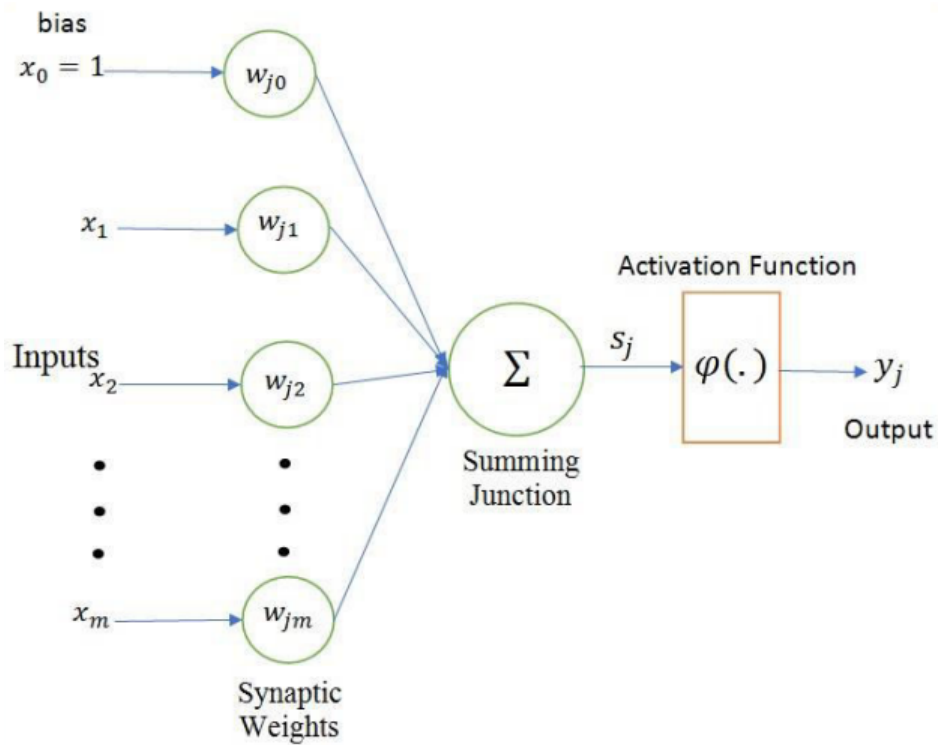


Figure 2.1 shows a simple neural network. A neural network consists of an input layer, which is a vector. Transforming the raw data (e.g. images, natural language, Java documentation, etc.) into a vector is known as *encoding*. How encoding is performed differs for all models, and is a consequential decision when designing a model. Common methods for encoding include: integer encoding, one-hot encoding, and word embeddings. Word embeddings are of particular importance as they are used by CodeBERT, the pre-trained model that will be used for this project.

Each neuron sums all the values of the neurons connected to it (generally, this is *all* the previous neurons.). The edges between neurons have *weights*, which are multiplied against the value before being added to the neuron. Furthermore, each neuron has a *bias* which is added to the neuron after each calculation. Figure 2.2 shows this graphically.

When this sum is generated, it is then passed into an *activation function*. The activation function stabilises the value of each neuron, ensuring they do not drop to zero or grow exponentially. These two issues are commonly referred to as the *vanishing gradient* and *exploding gradient* problems, respectively. The choice of activation function is important. There is a trade-off between the computational cost of activation functions, and how effective they are. Common activation functions include: the sigmoid function, hyperbolic tangent function, and gaussian function.

2.3 Training

Now that we understand the basics of what a neural network is, the most important aspect of deep-learning as it pertains to this thesis can be introduced — training. Training a neural network means to systematically adjust the values of the network (i.e, weights of edges between neurons and each neuron's bias) in order to generate more accurate results.

Initially, a neural network's weights and biases will be set a random. At this stage, any predictions made by the network are nothing more than guesses. However, if examples where the correct label is known are provided to the model, using algorithms such as *backpropagation*, the weights and biases of the network can be adjusted such that the new resulting output is now closer to the expected output. When this process done on many thousands of examples, with a proper splitting between train, test and evaluation datasets, the neural network is likely to have "learned" the general properties of whatever it is it is classifying, and therefore be able to accurately classify inputs it has not yet seen before.

2.4 Advanced Neural Networks

2.4.1 Recurrent Neural Networks

Processing language is difficult. The meaning of words is not just determined by itself, but also in what context the word is being used in. That is, the meaning of a word is not just determined by itself, but also by the words that surround it. The simple neural network introduced above would not be able to handle such a situation as it has no "memory" of previous inputs, i.e. it lacks contextual information.

Recurrent Neural Networks are one methodology for neural networks to have “memory” and thereby consider what context an input is being used in. Essentially, the state of an RNN at any input is determined by all previous inputs. Having “memory” of previous inputs allows RNNs to perform complex tasks on sequence data such as: voice recognition computer vision or natural language tasks such as translation.

While RNNs have historically proved useful for NLP tasks [13], they have numerous downsides. Perhaps the key issue with RNNs is that they are notoriously difficult to train, due to the *vanishing gradient* and *exploding gradient* problems. Difficulty here meaning both that they are computationally expensive to train, and can often produce inaccurate results for long sequences of inputs. Most solutions to these issues slow training considerably [14]. LSTM (Long short-term memory) networks are a class of RNN conceived to address the vanishing gradient and exploding gradient problems present in simple RNNs [15].

2.4.2 Long short-term Memory Networks

Long short-term memory networks (LTSMs) are a subclass of RNN designed to address the vanishing gradient and exploding gradient issues present in simple RNNs. Describing the precise methodology in which this is done is outside the scope of this report. However, put simply, this is achieved with “gating units” which are a more sophisticated method for calculating the hidden-state of a cell (Where the hidden-state of a cell is the “memory” the neural network uses to understand the context of an input [16]).

These gating units, in addition to using the activation function that simple RNNs use, have to calculate a variety of additional values. These extra computations accumulate and make LTSMs far more computationally intensive to train than their generic RNN counterparts. However, this is not without benefit. LTSMs are significantly more accurate for longer sequences compared to generic RNNs [17].

2.4.3 Transformers

The goal of the transformer model, introduced by Google Brain in 2017, was to handle sequential data, just as RNNs do, but without the training-time limitations. Ultimately, this goal was achieved as transformer models were demonstrated to perform significantly better than RNNs while simultaneously requiring less training time [18].

Long training times for RNNs are, ultimately, caused by the fact that input must be processed sequentially. Transformers, however, do not have this limitation. They can process multiple inputs simultaneously and those inputs do not have to be in order. It is because transformers are parallelisable

that they have quicker training times [18].

Chapter 3

Milestone 1

In order to learn how to fine-tune CodeBERT for a classification task, a simple milestone task was conducted. While the models generated in this milestone task do not directly pertain to the goals of this thesis, it is worth discussing as the methodology for performing this task is similar to how the project proper will be done.

3.1 Objective

The objective of this milestone task was to fine-tune CodeBERT for the simple binary classification task of determining if a pair of Javadoc and a Java method are related or not. I.e., *Does this Javadoc describe this method?* For example, Figure 3.1, below, shows an example of Javadoc and code match-

```
/**
return id of the session to identify a sessions in the sequence of all sessions
    for the same key
*/
public int getSessionId() {
    return sessionId;
}
```

Figure 3.1: An example of matching Javadoc and code

```
/**
Shutdown the executor so we don't leak threads into other test runs.
*/
public Collection<RetentionLease> leases() {
    return leases.values();
}
```

Figure 3.2: An example where Javadoc and code do not match

ing, whereas Figure 3.1 shows an example where the Javadoc and code do not match. While this task is trivial for humans, it is not simple for a neural network.

3.2 Data Collection

For CodeBERT to work this any degree of accuracy for this task, it needed to be fine-tuned with additional data. It was not clear how many examples exactly would be required but it was decided 30,000 examples would be collected, with a typical 80-10-10 split. That is, 24,000 training examples, 3,000 test examples and 3,000 evaluation examples.

The data was collected from numerous open-source projects (with the appropriate licensing) from Github, and examples were extracted using a tool programmed in Java. Exactly half the examples were positive — that is, the Javadoc does match the code. And, the other half of the examples were negative, where the Javadoc and code did not match. For positive examples, methods and their corresponding Javadoc were retrieved from the code-bases and combined together. For negative examples, a random Java method and random Javadoc were assigned together and assumed to be unrelated.

Since CodeBERT has a limited input size of 512 tokens, efforts were made to reduce the number of characters in the examples and remove as much superfluous information as possible. For example, embedded HTML tags (common in Javadocs) and newline characters were remove, and multiple spaces were replaced with a single space.

After this processing, if the combination of the Java method and the Javadoc was still over 512 tokens, the example was rejected. While this may initially seem to be an extreme solution, it was done from a lack of viable alternatives. One possibility would be to truncate the example, such that the example fit in 512 characters. However, upon further investigation, this is an unsuitable solution. Since we are working with bimodal data, if one truncates the example at the end, such that any tokens after 512 is ignored, then this unfairly biases the data towards the code segment of the data (as the input of CodeBERT is code first, then a separator token, followed by the natural-language). As an alternative to this, one could truncate an equal amount from the code and natural-language such that the final example is less than 512 tokens. However, this is also an inadequate solution as there is not a one-to-one relationship between Javadoc and code. That is, the first "x percent" of the Javadoc does not describe the first "x percent" of the code. As such, any attempt to truncate the examples this way would likely lead to spoiling the data and hence a less accurate model. There are additional techniques that can be employed to solve CodeBERT's maximum input length problem, and they will be discussed at a later stage of this thesis. For this milestone task, however, it was decided to simply reject these examples.

Chapter 4

Conclusion

Conclude your thesis.

Bibliography

- [1] F. Nielson, H. R. Nielson, C. Hankin, Principles of program analysis / Flemming Nielson, Hanne Riis Nielson, Chris Hankin., 1st Edition, Springer-Verlag, Berlin ; Heidelberg ; New York, 2005.
- [2] SonarSource, Sonarqube (2022).
URL <https://www.sonarqube.org/>
- [3] Spotbugs (2022).
URL <https://spotbugs.github.io/>
- [4] Snyk, Snykcode (2022).
URL <https://snyk.io/>
- [5] Facebook, Infer (2022).
URL <https://fbinfer.com/>
- [6] W. R. Nichols, The cost and benefits of static analysis during development (2020). doi:10.48550/ARXIV.2003.03001.
URL <https://arxiv.org/abs/2003.03001>
- [7] A. Møller, M. I. Schwartzbach, Static program analysis, Notes. Feb (2012).
- [8] B. Chess, G. McGraw, Static analysis for security, IEEE security & privacy 2 (6) (2004) 76–79.
- [9] F. Zampetti, S. Scalabrino, R. Oliveto, G. Canfora, M. Di Penta, How open source projects use static code analysis tools in continuous integration pipelines, in: 2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR), IEEE, 2017, pp. 334–344.
- [10] M. Rahimzadeh, A. Attar, S. M. Sakhaei, A fully automated deep learning-based network for detecting covid-19 from a new and large lung ct scan dataset, Biomedical Signal Processing and Control 68 (2021) 102588. doi:<https://doi.org/10.1016/j.bspc.2021.102588>.
URL <https://www.sciencedirect.com/science/article/pii/S1746809421001853>
- [11] L. Deng, Y. Liu, Deep learning in natural language processing / Li Deng, Yang Liu, editors., Springer, Singapore, 2018.

- [12] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, M. Zhou, Codebert: A pre-trained model for programming and natural languages (2020). doi: 10.48550/ARXIV.2002.08155.
URL <https://arxiv.org/abs/2002.08155>
- [13] D. Bahdanau, K. Cho, Y. Bengio, Neural machine translation by jointly learning to align and translate, in: Y. Bengio, Y. LeCun (Eds.), 3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings, 2015.
URL <http://arxiv.org/abs/1409.0473>
- [14] R. Pascanu, T. Mikolov, Y. Bengio, Understanding the exploding gradient problem, CoRR abs/1211.5063 (2012). arXiv:1211.5063.
URL <http://arxiv.org/abs/1211.5063>
- [15] S. Hochreiter, J. Schmidhuber, Long Short-Term Memory, Neural Computation 9 (8) (1997) 1735–1780. arXiv:<https://direct.mit.edu/neco/article-pdf/9/8/1735/813796/neco.1997.9.8.1735.pdf>, doi:10.1162/neco.1997.9.8.1735.
URL <https://doi.org/10.1162/neco.1997.9.8.1735>
- [16] F. Salem, Recurrent Neural Networks: From Simple to Gated Architectures, Springer, 2022.
URL <https://books.google.com.au/books?id=bJpXEAAAQBAJ>
- [17] M. Sundermeyer, R. Schlüter, H. Ney, Lstm neural networks for language modeling, in: Thirteenth annual conference of the international speech communication association, 2012.
- [18] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, CoRR abs/1706.03762 (2017). arXiv:1706.03762.
URL <http://arxiv.org/abs/1706.03762>

Appendix A

Appendix

Write your appendix here.